Singleton Design Pattern

What is the Singleton Pattern?

The Singleton is a **Creational Design Pattern** that ensures a class has only **one instance** while providing a global access point to that instance. It is one of the simplest patterns but requires careful implementation to handle multi-threading and security.

Real-World Use Cases

- **Logging:** Managing a shared log object to ensure all log messages across the system are written sequentially to a single file.
- **Database Connections:** Using a single connection pool object to manage a shared set of database connections, preventing resource exhaustion.
- **Configuration:** Storing global application settings (loaded once from a file) that need to be read consistently by various modules.
- **Caching:** Maintaining a global in-memory cache to store expensive computation results or database lookups.

1. Eager Initialization

```
public class EagerSingleton {

    // Instance is created at class loading time
    private static final EagerSingleton instance = new EagerSingleton();

    private EagerSingleton() {
        // Private constructor prevents external instantiation
    }

    public static EagerSingleton getInstance() {
        return instance;
    }
}
```

Pros

- Extremely simple to implement.
- **Implicitly Thread-Safe:** The JVM guarantees the instance is created before any thread accesses the static variable.

Cons (Interview Focus)

- **Resource Waste:** The instance is created during class loading, even if the client application never invokes `getInstance()`.

2. Lazy & Synchronized

```
public class LazySingleton {

    private static LazySingleton instance;

    private LazySingleton() {}

    // Synchronized Method ensures thread safety, but is extremely slow
    public static synchronized LazySingleton getInstance() {
        if (instance == null) {
            instance = new LazySingleton();
        }
        return instance;
    }
}
```

The Bottleneck: Using `synchronized` on the method level creates a massive performance hit. Every single call to `getInstance()` forces threads to wait for a lock, even after the instance has already been initialized.

3. Double-Checked Locking (DCL)

```
public class DCLSingleton {

    // 'volatile' is mandatory to prevent instruction reordering!
    private static volatile DCLSingleton instance;

    private DCLSingleton() {}

    public static DCLSingleton getInstance() {
        if (instance == null) { // 1st Check (No Locking overhead if already
initialized)
            synchronized (DCLSingleton.class) {
                if (instance == null) { // 2nd Check (Ensures only one thread
initializes)
                    instance = new DCLSingleton();
                }
            }
        }
        return instance;
    }
}
```

Deep Dive: Why 'volatile'?

Object creation (`instance = new DCLSingleton()`) is NOT an atomic operation. It involves three distinct steps at the bytecode level:

1. Allocate memory space for the object.
2. Execute the constructor to initialize the object.
3. Assign the memory address to the reference (`instance`).

Without `volatile`, the JVM/CPU is allowed to reorder steps 2 and 3 for optimization. If Step 3 happens before Step 2, Thread A assigns the memory address while the constructor is still running. Thread B then calls `getInstance()`, sees `instance != null`, and returns a **partially-initialized** object, leading to a system crash. `volatile` establishes a "happens-before" guarantee, preventing this reordering.

4. Bill Pugh Implementation (Holder Pattern)

```
public class BillPughSingleton {

    private BillPughSingleton() {}

    // Inner static class is loaded ONLY when getInstance() is called
    private static class SingletonHolder {
        private static final BillPughSingleton INSTANCE = new
BillPughSingleton();
    }

    public static BillPughSingleton getInstance() {
        return SingletonHolder.INSTANCE;
    }
}
```

Mechanism: Prior to Java 5, the memory model made DCL unreliable. Bill Pugh proposed the Static Inner Class approach. The `SingletonHolder` class is not loaded into memory until `getInstance()` is invoked. This achieves 100% thread-safe lazy initialization **without** the overhead of explicitly using synchronization blocks.

5. Breaking Singleton: Reflection

Attackers or poorly written code can use Java Reflection to bypass the private constructor.

```
// The Attack:
Constructor[] constructors = Singleton.class.getDeclaredConstructors();
for (Constructor c : constructors) {
    c.setAccessible(true); // Disables access checks!
    Singleton brokenInstance = (Singleton) c.newInstance();
}
```

The Defense: Throw an exception inside the private constructor if an instance already exists.

```
private Singleton() {
    if (instance != null) {
        throw new RuntimeException("Instance already exists. Use getInstance()
method.");
    }
}
```

6. Breaking Singleton: Serialization

If a Singleton implements `Serializable`, deserializing it will create a brand new instance, bypassing the constructor entirely.

The Defense: You must provide an implementation for the special `readResolve()` method.

```
protected Object readResolve() {
    // The JVM will substitute the newly deserialized object with this returned instance
    return getInstance();
}
```

7. The Enum Solution (Gold Standard)

```
public enum EnumSingleton {

    INSTANCE;

    public void executeBusinessLogic() {
        // Code goes here
    }
}
```

Why is this the absolute best approach?

- **Reflection Proof:** The Java Language Specification explicitly prevents instantiation of Enums via reflection. `Constructor.newInstance()` will throw an `IllegalArgumentException`.
- **Thread Safe:** Enum values are instantiated by the JVM when the class is loaded, guaranteeing thread safety natively.
- **Serialization Safe:** The JVM handles Enum serialization automatically, ensuring that deserialization always returns the exact same constant.

Trade-off: Enums are strictly eagerly initialized and cannot implement lazy loading. Furthermore, they cannot extend other classes (as they inherently extend `java.lang.Enum`).

Singleton Pattern vs Static Utility Class

Feature	Singleton Pattern	Static Class
OOP Principles	Can implement interfaces, extend classes, and support polymorphism.	Procedural approach. Cannot inherit or implement interfaces.
State	Designed to maintain an internal state safely across the application.	Best used for stateless utility functions (e.g., `Math.abs()`).
Testing / Mocking	Easier to mock out in unit tests (especially if passed via Dependency Injection).	Extremely difficult to mock static methods.
Memory	Can be lazily loaded to save heap space until absolutely required.	Loaded as soon as the class is referenced.